

CSC 365 Lab 1: Why Databases?

Team Members

Sage Kaplan

Initial Decisions

I chose Python 3 as the programming language for this lab since it's the language I'm most familiar with and has file handling, parsing, object-oriented programming, and command-line input without requiring additional packages.

The program was developed and tested from the terminal in VS Code in Python 3. The program does not require compilation or any external libraries.

Internal Architecture

The program is organized around a Student class. Each student record from students.txt is represented as one Student object containing the student's last name, first name, grade, classroom, bus route, GPA, teacher last name, and teacher first name.

When the program starts, `parse_students()` opens students.txt, separates each line into its eight fields, converts the numeric fields to the appropriate numeric types, creates a Student object, and stores the object in a list.

The list of Student objects is kept in memory while the program is running. The `receive_input()` function repeatedly prompts the user for a command and searches through this list for matching records.

The program supports student, teacher, bus, grade, GPA high/low, average GPA, and grade information searches. Since the data set is relatively small, each command performs a linear search through the list rather than using additional indexing structures.

The parser also checks for a missing/incorrect students.txt file, an incorrect number of fields, and invalid numeric data. Invalid search commands return the user to the input prompt without terminating the program.

Task Log

Task: Design program architecture and Student class

Student: Sage Kaplan

Start Time: Friday 8/28 1 pm

End Time: Friday 8/28 1:30 pm

Person-Hours: 30 min

Task: Implement students.txt parser and file error handling

Student: Sage Kaplan

Start Time: Tuesday 9/1 5 pm

End Time: Tuesday 9/1 8 pm

Person-Hours: 3 hours

Task: Implement command input loop and search commands

Student: Sage Kaplan

Start Time: Tuesday 9/1 8 pm

End Time: Wednesday 9/2 1 am

Person-Hours: 5 hours

Task: Implement GPA, average, and Info commands

Student: Sage Kaplan

Start Time: Tuesday 9/1 8 pm

End Time: Wednesday 9/2 1 am

Person-Hours: 5 hours

Task: Test and debug program

Student: Sage Kaplan

Start Time: Wednesday 9/2 1:30 pm

End Time: Wednesday 9/2 2:30 pm

Person-Hours: 1 hour

Task: Prepare test suite, README, and submission files

Student: Sage Kaplan

Start Time: Wednesday 9/2 2:30 pm

End Time: 3:30

Person-Hours: 1 hour

Testing Notes

Testing was performed by me (Sage Kaplan) on September 2, 2026.

The test suite covers the student, student bus, teacher, bus, grade, high GPA, low GPA, average GPA, Info, and Quit commands. Testing also includes invalid input, a valid search with no matching results, and execution without students.txt.

Two command-validation issues were found during testing. The student search originally accepted an invalid second argument, such as S: HAVIR X, as a normal student search. The grade search similarly accepted an invalid GPA option, such as G: 4 X, as a normal grade search. Checks were added so that the student command only accepts the optional B or Bus argument and the grade command only accepts H, High, L, or Low.

Total testing time: 1 hour

Time spent fixing bugs: 1 hour

The final test run produced the expected results for the tested commands. Testing bugs and fixing bugs were intertwined so the timing will be grouped together.

Final Notes

I didn't really track how much time I spent doing certain parts and when I did them, so I used my GitHub commits and memory to estimate for task timings. Also, I would test and debug as I wrote sections, so it's hard to estimate and put a time frame for those as well.